

Computer Architecture Project Report - Dr. Jahangir

Nader Ahmadi - 403105674

Sharif University of Technology - 2026

Notes

An **Apple M2 processor** was used to perform the experiments in this project.

1 Part 1: Extracting Microarchitectural Fingerprints via Timing

In this section, we aim to investigate the following parameters:

- Cache levels and sizes (L1, L2, L3)
- Cache Line Size
- Degree of Set Associativity (N-way Set-Associative)

1.1 Cache Level Sizes

In this part, we attempt to measure the latency of different cache levels using working dataset sizes ranging from 4 KB to 64 MB. These sizes are doubled in each iteration to accurately detect the boundaries between different cache levels.

First, an array of nodes is created, where the size of each node is 128 bytes. The number of nodes at each test step is calculated by dividing the total data size by the size of a single node. The choice of 128 bytes ensures complete filling of a cache line and prevents node misalignment across two separate cache blocks. A size of 64 bytes was also tested, yielding identical results due to alignment maintenance.

The connection between nodes is implemented using the **Pointer Chasing** technique, where each node contains the address of the next node, forming a sequential chain. Furthermore, to disable the hardware prefetcher and prevent the CPU from discovering access patterns, the **Fisher-Yates Shuffle** algorithm is employed. By traversing from the end of the array to the beginning, this algorithm shuffles the **next** pointers among various nodes.

```
void shuffle_nodes(struct Node* array, size_t count) {
    for (size_t i = 0; i < count; i++) {
        array[i].next = &array[(i + 1) % count];
    }
    srand((unsigned int)time(NULL));
    for (size_t i = count - 1; i > 0; i--) {
        size_t j = rand() % i;
        struct Node* temp = array[i].next;
        array[i].next = array[j].next;
        array[j].next = temp;
    }
}
```

Figure 1: Fisher-Yates Shuffle

Since traversing the chain is extremely fast and makes execution time measurement difficult, we repeat the traversal 10 million times per loop iteration to produce a measurable execution time.

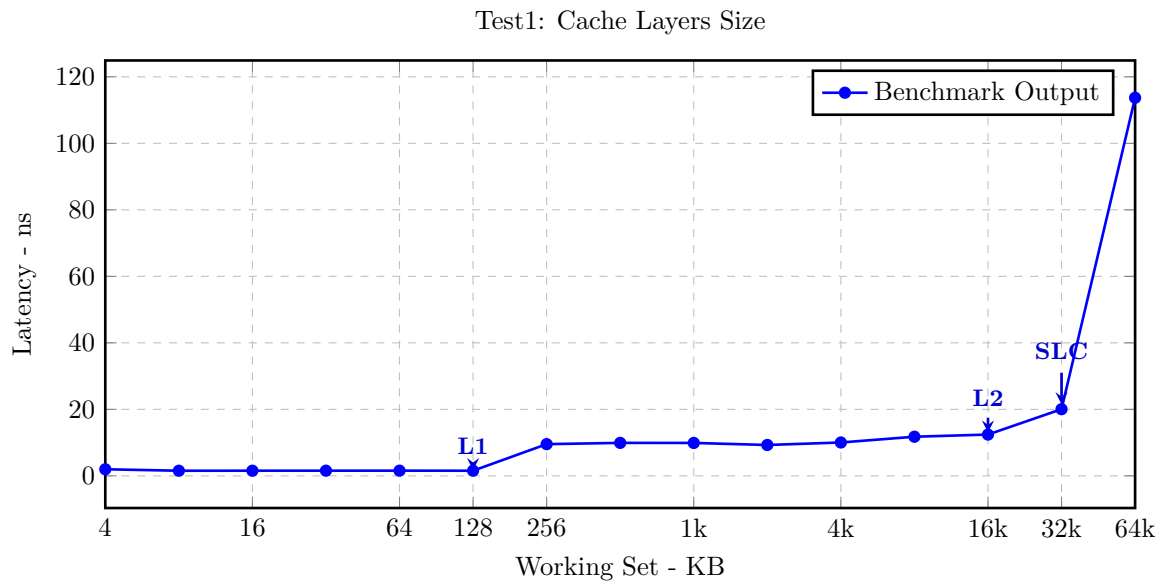
The output of this step is presented below.

```

=== TEST 1: Extracting Cache Levels (Cache Sizes) ===
Working Set (KB)    Num Nodes    Latency (ns)
-----
4.00                32           1.980
8.00                64           1.538
16.00               128           1.551
32.00               256           1.555
64.00               512           1.562
128.00              1024          1.543
256.00              2048          9.534
512.00              4096          9.913
1024.00             8192          9.895
2048.00             16384         9.284
4096.00             32768        10.033
8192.00             65536        11.763
16384.00            131072       12.418
32768.00            262144       20.054
65536.00            524288       113.705
>> TEST 1 Total Execution Time: 2555.399 ms (2.555 s)

```

Figure 2: Cache Layer Size Test



Results

As can be observed, up to 128 KB, the data fetch latency remains around **1.5 ns**, which corresponds to the **Level 1 (L1) Cache**. From 256 KB to 16 MB, the latency stays between **9.5 ns** and **12 ns**, representing the **Level 2 (L2) Cache**. Beyond that, a significant latency spike indicates main memory (RAM) access.

Note: The M2 processor series does not feature a traditional L3 cache; instead, it incorporates a System Level Cache (SLC) shared across different functional blocks of the SoC. The capacity range of this cache is evaluated between 16 MB and 32 MB.

Next, we compare our findings with the ground truth hardware values obtained via the following terminal command:

```
sysctl hw.perflevel0.l1dcachesize hw.perflevel0.l1icachesize
hw.perflevel0.l2cachesize
```

The output of this command is:

```
hw.perflevel0.l1dcachesize: 131072
hw.perflevel0.l1icachesize: 196608
hw.perflevel0.l2cachesize: 16777216
```

which closely matches our experimental benchmark measurements.

Occasionally, slight variations in the exact latency jump boundaries may occur across different runs. This happens because a portion of the L1 cache capacity may be occupied by background OS processes during execution.

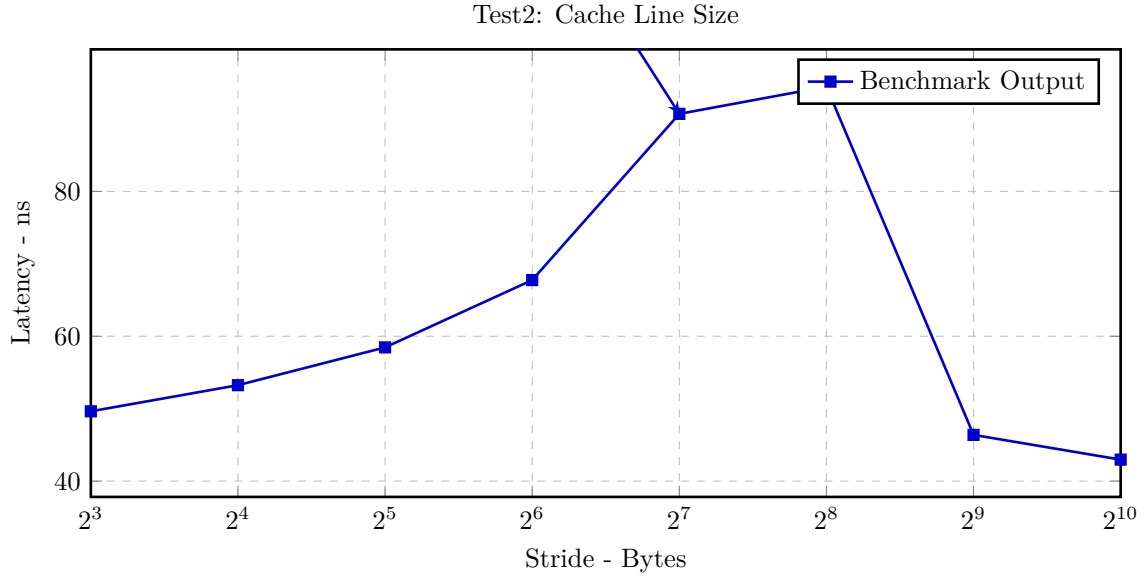
1.2 Cache Line Size

In this section, the goal is to determine the exact size of each cache line (block) by varying the traversal stride (**Stride**). To perform this test, a data array of **32 MB** is allocated so that the dataset exceeds the L2 cache size and resides in **RAM**, making memory access latency clearly measurable.

At each step, the total dataset size is divided by the **Stride** value to determine the number of nodes in the chain. Increasing the stride naturally reduces the total number of traversable nodes per cycle.

```
=== TEST 2: Extracting Cache Line Size ===
Stride (Bytes)      Latency (ns)
-----
8                   49.632
16                  53.243
32                  58.460
64                  67.753
128                 90.703
256                 94.460
512                 46.379
1024                42.963
>> TEST 2 Total Execution Time: 7673.655 ms (7.674 s)
```

Figure 3: Cache Line Size Test



Results

As shown in Figure 3, a steep jump in access latency occurs at a **stride of 128 bytes**. Thus, we can conclude that the **Cache Line Size** is **128 bytes**. The decrease in latency at larger strides (256 bytes and above) is attributed to the drastic reduction in total node count, which significantly decreases total memory accesses per run.

To verify this result, the following command was executed in the terminal:

```
sysctl hw.cachelinesize
```

The command returned:

```
hw.cachelinesize: 128
```

confirming the accuracy of our benchmark observation.

1.3 Set Associativity

In this section, the objective is to determine the number of cache ways (**Ways**) that can simultaneously map to a specific set (**Set**) in the cache. To achieve this, data points are selected at a constant offset of 16 KB apart, ensuring that all selected addresses map to the exact same cache set.

By incrementally increasing the number of mapped addresses from 1 to 16, we evaluate the capacity of that set. As long as the number of data items does not exceed the hardware set associativity, all items fit within the set, maintaining minimal latency. However, once set capacity is exceeded, additional items trigger **Conflict Misses**, evicting existing lines and causing a distinct latency spike that reveals the exact degree of associativity.

The benchmark output for this step is as follows:

Results

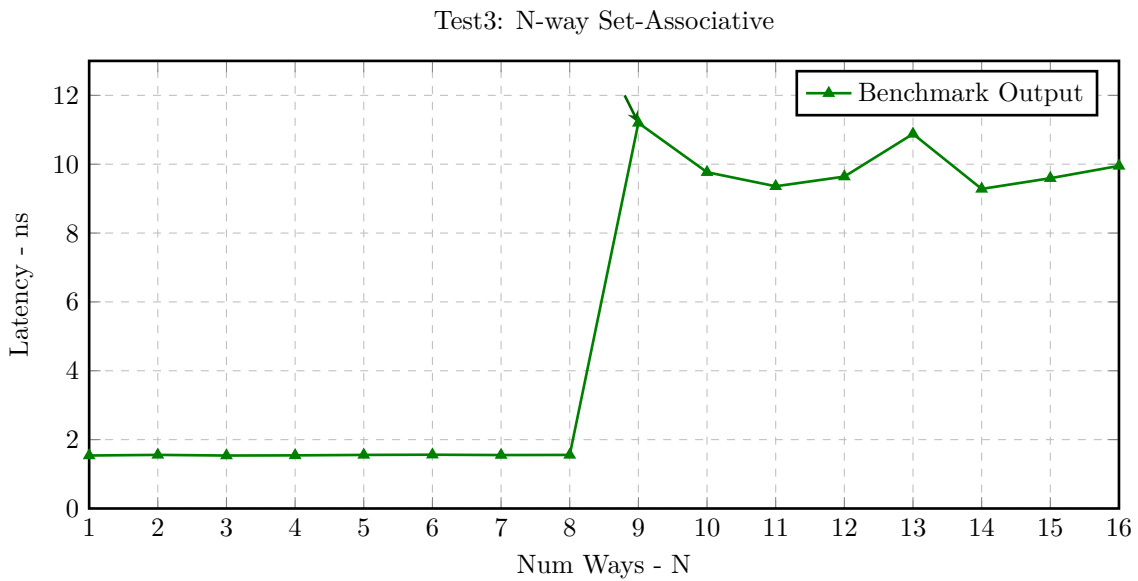
As seen in the graph, a sudden spike in latency occurs at $N = 9$. We can therefore conclude that the processor features an **8-way set-associative L1 cache**. While official technical documentation from Apple regarding set associativity is limited, standard microarchitectural literature and LLM architectural references confirm that the Apple Silicon L1 Data Cache is indeed 8-way set-associative.

```

=== TEST 3: Estimating Cache Associativity ===
Num Ways (N)      Latency (ns)
-----
1                  1.539
2                  1.557
3                  1.538
4                  1.543
5                  1.556
6                  1.562
7                  1.552
8                  1.556
9                  11.200
10                 9.767
11                 9.359
12                 9.640
13                 10.880
14                 9.282
15                 9.593
16                 9.951
>> TEST 3 Total Execution Time: 920.951 ms (0.921 s)

```

Figure 4: Set Associativity Test



2 Part 2: Reading Hardware Performance Counters

Notes

Since the profiling tools mentioned in the generic project description were not directly available for macOS, we utilized **Xcode Instruments**. This tool allows detailed monitoring of CPU microarchitectural hardware events. To profile hardware performance counters, the target executable was attached within Instruments, adding custom event profiling configurations. The profiling session output is saved in the native **Trace** format.

The benchmark executable developed in Part 1 was profiled using this setup. The extracted microarchitectural metrics are summarized in the table below:

The breakdown of function execution times is detailed below:

2.1 Correlating Profiling Data with Part 1

We now analyze the individual timeline phases:

Table 1: Microarchitecture Metrics Extracted from Xcode Instruments for Process (Samples: 141,538)

Microarchitectural Metric	Hardware Event	Total Count	Calculated Rate / Ratio
Total Executed Instructions	INST_ALL	6,443,573,748	100.00%
Total CPU Clock Cycles	Cycles	25,523,723,774	–
Execution Efficiency (IPC)	INST_ALL / Cycles	–	0.2525 (CPI = 3.96)
L1 Data Cache Miss (Load)	L1D_CACHE_MISS_LD	295,389,509	–
L1 Data Cache Miss (Store)	L1D_CACHE_MISS_ST	23,571,344	–
Total L1 Data Cache Misses	L1D_CACHE_MISS (Total)	318,960,853	4.95% (of total instructions)
L1 Data TLB Accesses	L1D_TLB_ACCESS	3,533,290,451	–
L1D Data TLB Misses	L1D_TLB_MISS	163,779,190	4.64% (TLB Miss Rate)
Branch Instructions	INST_BRANCH	1,811,420,549	28.11% (of total instructions)
Branch Mispredictions	BRANCH_MISPRED_NONSPEC	18,873	0.0010% (Misprediction Rate)
Profiling Resolution	# Samples	141,538	$\approx 10,011$ Hz (Samples/sec)

Table 2: Function Execution Time Distribution (Time Profiler Breakdown)

Benchmark Function	Total Time (Weight)	Self Time	% of Total Runtime
run_cache_line_test	10.52 s	10.34 s	74.4%
run_cache_size_test	2.67 s	2.63 s	18.9%
run_associativity_test	948.20 ms	948.10 ms	6.7%
Total Execution Time	14.14 s	–	100.0%

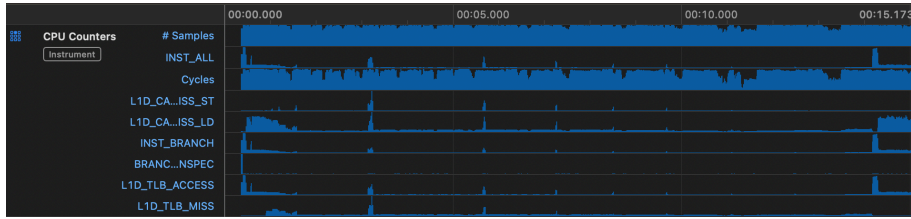


Figure 5: Overall Program Execution Timeline in Xcode Instruments

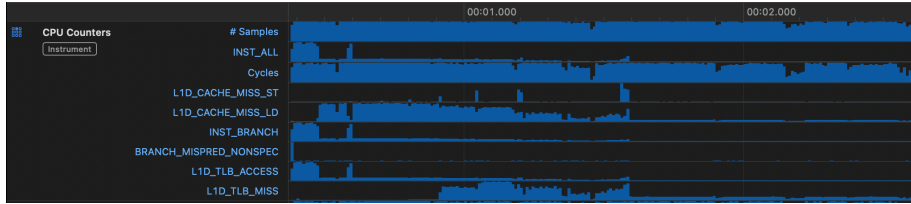


Figure 6: Timeline for Cache Layer Size Test

As observed, L1 data cache misses remain low initially. However, as dataset sizes increase beyond L1 bounds, miss counts experience a sharp surge, perfectly aligning with the findings in Part 1. Furthermore, towards the end of this test phase, miss density decreases. This occurs because at large array sizes (e.g., **16 MB**, **32 MB**), each cache miss incurs a significant RAM latency delay, stalling the processing pipeline and resulting in fewer total misses generated per unit of time.

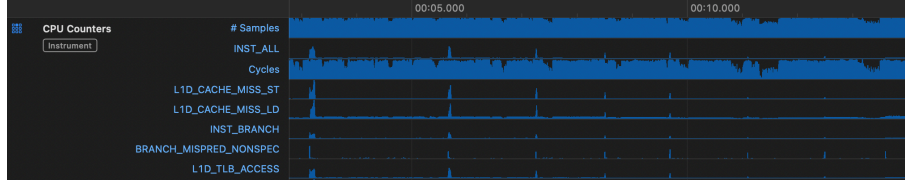


Figure 7: Timeline for Cache Line Size Test

The timeline exhibits distinct periodic spikes while maintaining relatively uniform miss rates in between. These spikes correspond to the initialization and data shuffling (Fisher-Yates) phases at the start of each stride iteration. In earlier steps with smaller stride values, the number of nodes is substantially higher, leading to denser profiling spikes.

Between these setup spikes, cache miss rates remain stable. Because the working dataset is fixed at **32 MB** (exceeding L2 cache size), fetching data from RAM introduces fixed memory latency bottlenecks, keeping miss density constant. Additionally, the interval between spikes decreases as the stride increases due to the reduction in node count and initialization overhead.



Figure 8: Timeline for Set Associativity Test

At lower associativity degrees, all accessed addresses fit within a single cache set, yielding very few cache misses. Once the number of concurrent pointers exceeds the degree of set associativity ($N > 8$), items can no longer coexist in the set simultaneously. This triggers continuous eviction cycles and a dramatic surge in cache misses.

3 Part 3: Analysis and Optimization of a Performance Phenomenon

Investigated Phenomenon: Row-Major vs. Column-Major Matrix Traversal

3.1 Hypothesis

Since multi-dimensional array elements are stored contiguously in memory in row-major order (C-style), row-major traversal is expected to exhibit lower cache miss rates and execution latency compared to column-major traversal, owing to spatial locality.

3.2 Experiment and Analysis

To evaluate this phenomenon, a test program was created using a 4096×4096 matrix of 4-byte integers. Row-major traversal computes `matrix[i * SIZE + j] += i + j;` by iterating over rows in the outer loop, whereas column-major traversal iterates over columns in the outer loop.

The snippet corresponding to column-major traversal is shown below:

```

for (int j = 0; j < SIZE; j++) {
    for (int i = 0; i < SIZE; i++) {
        matrix[i * SIZE + j] += i + j;
    }
}

```

Figure 9: Column-Major Traversal Code Snippet

The experimental results from the command line interface and Instruments are presented below:

```

=== TEST 1: Row-Major Matrix Traversal ===
Row-Major Traversal Time: 27.483 ms
>> TEST 1 Total Execution Time: 64.351 ms

=== TEST 2: Column-Major Matrix Traversal ===
Column-Major Traversal Time: 168.776 ms
>> TEST 2 Total Execution Time: 199.452 ms

=====
>> TOTAL PROGRAM EXECUTION TIME: 263.810 ms (0.264 s)
=====

```

Figure 10: Command Line Benchmark Output

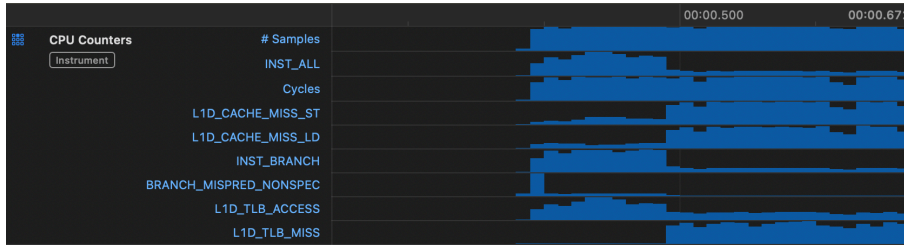


Figure 11: Performance Counter Profile in Xcode Instruments

The column-major traversal exhibits drastically higher execution time and L1 data cache miss rates compared to row-major traversal.

This behavior is directly explained by the principles of spatial locality in cache design. When a cache line is loaded, adjacent memory addresses are fetched into the cache simultaneously. In row-major traversal, contiguous memory accesses benefit from high hit rates because adjacent row elements reside within the same cache line. Conversely, column-major access straddles distinct memory rows (separated by large stride offsets), missing cached data on almost every access and inflating miss density.

3.3 Ruling Out Alternative Hypotheses: Branch Mispredictions

One might hypothesize that branch misprediction overhead accounts for the latency disparity in column-major traversal. However, analysis of hardware branch counters reveals that branch mispredictions occur almost exclusively during loop setup and remain negligible throughout execution (28,546 mispredictions out of > 260M branch instructions). Thus, branch prediction performance is ruled out as a factor.

3.4 Optimization Strategy: Cache Tiling

To mitigate cache misses in column-major traversal, we apply **Cache Tiling** (blocking). By partitioning the matrix into small sub-blocks of size 32×32 , traversal is confined locally within each tile. Since each

Table 3: Hardware Counters - Unoptimized (Column-Major)

Metric	Samples	INST_ALL	Cycles	MISS_ST	MISS_LD	INST_BRANCH	MISPRED_NONSPEC	TLB_ACCESS	TLB_MISS
Total	2747	1,109,406,473	516,911,846	20,379,099	20,228,561	260,724,223	28,546	690,529,492	128,591,428

tile consumes only 4 KB of memory (well within L1 capacity), data fits comfortably in cache, dramatically reducing latency.

The hardware counter metrics after applying loop tiling are detailed below:

Table 4: Hardware Counters - Optimized (Tiled)

Metric	Samples	INST_ALL	Cycles	MISS_ST	MISS_LD	INST_BRANCH	MISPRED_NONSPEC	TLB_ACCESS	TLB_MISS
Total	2208	1,157,513,145	418,807,471	10,803,379	25,678,832	265,005,114	42,767	661,948,878	4,542,589

Below is a direct comparison between the original and optimized implementations:

Table 5: Hardware Counter Metrics Comparison

Hardware Metric	Original (Unoptimized)	Optimized (Tiled)	Improvement / Change
Execution Time (ms)	271.60	217.20	-20.0% (Faster)
Samples	2,747	2,208	-19.6%
Total INST_ALL	1,109,406,473	1,157,513,145	+4.3% (Overhead)
Total Cycles	516,911,846	418,807,471	-19.0% (Faster)
L1D_CACHE_MISS_ST	20,379,099	10,803,379	-47.0% (Reduced)
L1D_CACHE_MISS_LD	20,228,561	25,678,832	+26.9%
INST_BRANCH	260,724,223	265,005,114	+1.6%
BRANCH_MISPRED_NONSPEC	28,546	42,767	+14,221 (< 0.01% total)
L1D_TLB_ACCESS	690,529,492	661,948,878	-4.1%
L1D_TLB_MISS	128,591,428	4,542,589	-96.5% (Massive Drop)

3.5 Cross-Sectional Synthesis

Based on Part 1 results, the processor cache line size is 128 bytes, meaning each cache line fill fetches 128 bytes into L1 cache. In unoptimized column-major traversal of a 4096×4096 matrix, traversing a single column spans 4096×128 bytes = 512 KB. Given that the processor's L1 Data Cache capacity is 128 KB, traversing a column completely evicts earlier cache lines before they can be reused, inducing severe thrashing. Tiling prevents this eviction cycle, resulting in a ****96.5% reduction in TLB misses**** and a ****20% speedup in execution time****.